

FLAWFIX SECUREASSIST

Automated Vulnerability Detection, Semantic LLVM IR Analysis & Patch Verification Audit Report

Target File: ctest.c	Language: C
Project: FlawFix Workspace	Audit Date: 2026-09-14 15:28:31
Report ID: RPT-15FBE792	Engine: FlawFix LLVM+XAI v1.0

1. Executive Security Assessment

The program contains two classic stack-based buffer overflow vulnerabilities. The first occurs in processInput due to the use of scanf with an unbounded %s format specifier. The second occurs in displayUser where strcpy copies a potentially larger buffer into a smaller local buffer without bounds checking.

TOTAL FLAWS	CRITICAL	HIGH	MEDIUM	LOW
2	0	2	0	0

2. Detected Vulnerability Inventory

ID	Classification (CWE)	Severity	Function	Lines
VULN-001	CWE-120: Buffer Copy without Checking Size of Input Stack-based Buffer Overflow via Unbounded scanf	HIGH	@processInput	18
VULN-002	CWE-120: Buffer Copy without Checking Size of Input Stack-based Buffer Overflow via Unbounded strcpy	HIGH	@displayUser	7

3. Explainable AI (XAI) Diagnostic Details

Finding 1: Stack-based Buffer Overflow via Unbounded scanf (CWE-120: Buffer Copy without Checking Size of Input)

Root Cause: The scanf function is called with the '%s' format specifier to read user input into the 'name' buffer of size 100. Because '%s' does not enforce any length restrictions, an attacker can input a string longer than 99 characters, overwriting adjacent stack memory, including the function's return address.

Security Exploit Impact: An attacker can exploit this vulnerability to overwrite stack frames, leading to a Denial of Service (crash) or arbitrary code execution with the privileges of the running process.

Remediation Guideline: Replace the unbounded scanf call with a bounded alternative, such as specifying a maximum width in the format string (e.g., '%99s') or using fgets() which safely respects buffer limits.

AI-Generated Secure Patch: *Bounded Input using fgets*

```
void processInput() {
    char name[100];

    printf("Enter your name: ");
    if (fgets(name, sizeof(name), stdin) != NULL) {
        name[strcspn(name, "\n")] = '\0';
    } else {
        name[0] = '\0';
    }

    displayUser(name);
}
```

Optimization Notes: Using fgets ensures that no more than 99 characters are read, preventing buffer overflows while preserving space for the null terminator.

Finding 2: Stack-based Buffer Overflow via Unbounded strcpy (CWE-120: Buffer Copy without Checking Size of Input)

Root Cause: The displayUser function copies the 'name' parameter (which can be up to 99 characters or more) into a local 'username' buffer of size 20 using strcpy. Since strcpy does not check the destination buffer's boundaries, any input longer than 19 characters will overflow the stack.

Security Exploit Impact: This allows stack corruption, leading to application crashes or potential control-flow hijacking if the return address is overwritten.

Remediation Guideline: Use bounded string copying functions such as snprintf or strncpy (with manual null-termination) to ensure the destination buffer is never exceeded.

AI-Generated Secure Patch: *Safe Copy using snprintf*

```
void displayUser(char *name) {
    char username[20];

    snprintf(username, sizeof(username), "%s", name);

    printf("User Profile\n");
    printf("=====\n");
    printf("Username: %s\n", username);
}
```

Optimization Notes: snprintf automatically truncates the input if it exceeds 19 characters and guarantees null-termination, preventing memory corruption.

5. AI Code Optimization Recommendations

- **Const Correctness:** Mark the 'name' parameter in displayUser as 'const char *' to indicate that the function does not modify the source string. (*Impact: Improves code readability, safety, and allows compiler optimizations.*)